Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

…with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu[1] Dung Van Bui[2] Nhut Tri Do[2] Van Du Nguyen[3]

FAIR 2026

[1]University of Transport and Communications, HCMC Campus [2]University of Information Technology, VNU-HCM
[3]Nong Lam University, HCMC

I. PROBLEM AND CONTRIBUTIONS

A near-perfect F1 on CICIDS2017 is a warning sign

Two problems, usually treated apart

- **Cost.** ≈80 features per flow — expensive to train, to score, to fit on an edge device.
- **Trust.** Near-perfect binary F1 is normally **data leakage**.

Two concrete leakage sources

1. `destination_port` encodes the *attacked service* → a port → label shortcut.
2. **Feature-level duplicates** survive full-row dedup → the same sample lands in train *and* test.

This paper

Compare four reduction strategies over the same eight Spark ML classifiers — but first make the *evaluation pipeline* trustworthy.

The contribution is the protocol

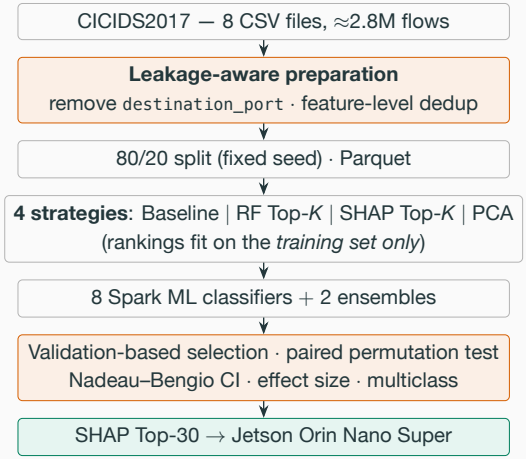
Not “which method wins” but **how to measure so the answer means something**.

Contributions

1. A **uniform comparison** of RF Importance, SHAP and PCA across the same 8 Spark ML algorithms + 2 ensembles on CICIDS2017.
2. SHAP contrasted **directly** against RF Importance on the same training set.
3. A **leakage-aware, statistically tested** pipeline: leak removal, deterministic feature-level dedup, validation-based selection, exact paired permutation test with Nadeau–Bengio correction.
4. A **per-attack-type multiclass** evaluation — where the methods genuinely differ.
5. An **edge-aware** recommendation: F1 *and* feature count *and* model size, for a Jetson Orin Nano Super (deployed in a companion systems paper).

II. LEAKAGE-AWARE PIPELINE

The leakage-aware pipeline



Removals happen *before* the split

- Feature-identical groups with **conflicting labels** are dropped *entirely*: **270 groups, 44,260 rows**.
- Result: 77 features, 1.79M / 0.45M train/test flows.

Design fixed *a priori*

- Exploration (*K* sweep) separated from confirmation.
- Hyper-parameters shared across all methods.
- The test set enters **no** selection step.

III. RESULTS AND CONCLUSION

Result 1: 60% of the features are removable

Method	Model	F1	Train (s)	Feat.
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	**0.9970**	**1226.1**	**30**
PCA k=40	XGBoost	0.9939	1444.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

Algorithm	RF Top-30	SHAP Top-30	Δ
Random Forest	0.9879	0.9955	+**0.77%**
XGBoost	0.9922	0.9970	+**0.48%**
GBT	0.9844	0.9918	+**0.75%**
Decision Tree	0.9835	0.9948	+**1.15%**

- SHAP Top-30 keeps baseline F1 while cutting ≈**60% of features** and ≈**73% of training time**.
- At *equal* feature count, SHAP beats RF Importance on every tree-based algorithm.
- PCA uses k=40 because it *extracts* rather than *selects* — each component carries only part of the variance. Cheaper, but not explainable.

Result 2: testing the difference instead of asserting it

Method	Model	F1 (mean ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009

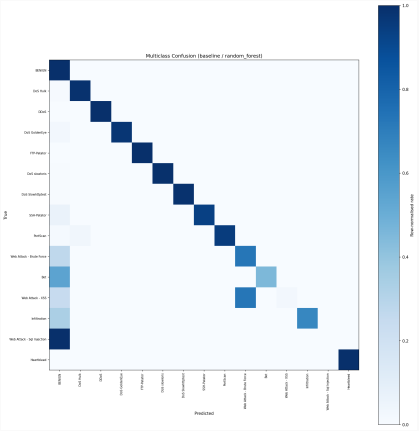
paired permutation $p = 0.031$; $d \approx 1.94$

Port ablation (RF)	F1	Recall (Attack)
Keep port	0.9965	0.9988
Remove port	0.9955	0.9961

- N=6 paired *resamplings*: variance reflects data sampling, not just seeds.
- Two-sided permutation test, **exact enumeration** of all 2^6 sign flips: smallest reachable p is $2/2^6$, so $N \geq 6$ is *required* to cross 0.05.
- Overlapping resamplings break t-independence $\rightarrow$ we use the **Nadeau–Bengio corrected** CI.

Significant *only at the resolution floor*, absolute gap ≈0.0001: the two are **practically equivalent**, so explainability and cost decide. After dedup, the port's isolated effect is small — the duplicates were the dominant leak.

Result 3: binary F1 hides everything that matters



Class	Support	Recall	F1
Benign	380,119	0.9997	0.9989
DoS Hulk	34,446	0.9904	0.9947
Web – Brute Force	311	0.7299	0.7323
Bot	290	**0.4552**	0.6256
Web – XSS	111	**0.0270**	0.0526
Web – SQL Injection	6	**0.0000**	0.0000
weighted-F1 = 0.998			**macro-F1 = 0.806**

Two failure modes, read *by destination*:

- **Mislabelled but caught**: 81/111 XSS land on Web Brute Force — **76% still flagged** at the binary level.
- **Leaking into Benign** (dangerous): all 6 SQL Injection, **54% of Bot**, 27% of Brute Force.

Result 4: the most honest number in the paper

Train	Test	F1
CICIDS2017	CICIDS2017	**0.9952**
CICIDS2017	CSE-CIC-IDS2018	**0.0423**
CSE-CIC-IDS2018	CSE-CIC-IDS2018	**0.9245**
CSE-CIC-IDS2018	CICIDS2017	**0.0535**

Shared leakage-free feature set;
CSE-CIC-IDS2018 label-stratified sample
(∼2.39M flows after dedup, fixed seed).

- In-domain F1 is high in *both* directions — cross-dataset F1 collapses to **0.04–0.05**.
- Driven by near-zero *recall* (0.022 / 0.030) while precision stays at 0.596: the model **fails to recognise** the other testbed's attacks rather than alarming indiscriminately.
- Causes: different CICFlowMeter version, different network configuration, non-overlapping attack tooling.

Near-perfect in-domain scores — *even leakage-free ones* — do not certify deployability under distribution shift.

Conclusion

What we showed

- SHAP Top-30 retains baseline F1 with **≈60% fewer features**.
- Baseline vs. SHAP Top-30 are **practically equivalent**.
- **macro-F1 0.806** against weighted-F1 0.998.
- Cross-dataset F1 **collapses to 0.04–0.05**.

The real contribution

- Not “which method is best” but **how to evaluate reliably**.

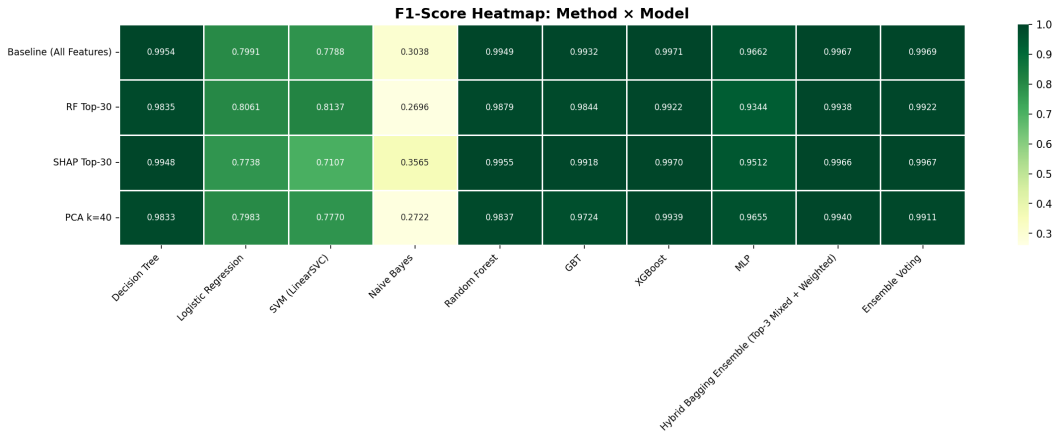
Next

- Full CSE-CIC-IDS2018; newer sets (RoEduNet, CIC-IoT).
- Does the SHAP Top-30 *set itself* transfer?
- Targeted imbalance handling; temporal features for beaconing.

Thank you — questions?

Backup slides

Backup: F1 heatmap, method × algorithm



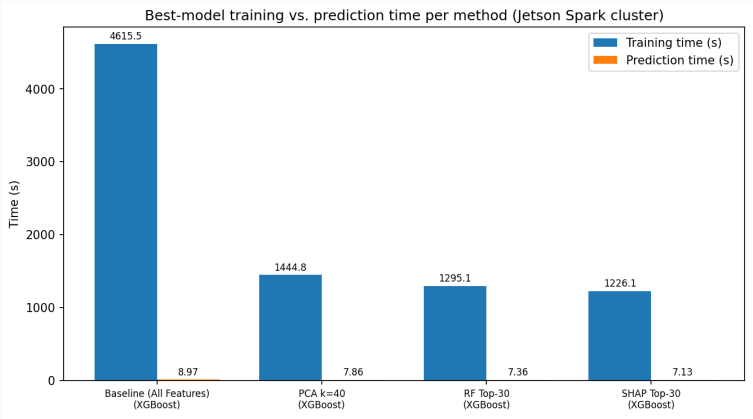
Binary, port removed. 8 algorithms × 4 methods.

Backup: hyper-parameters, fixed a priori

Model	Main hyper-parameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[*d*,64,32,2], maxIter=150, stepSize=0.01

Shared across all reduction methods, so differences reflect the *method*, not per-model tuning. Class weights for 6 models, `scale_pos_weight` for XGBoost; MLP unweighted (Spark ML has no sample weights for it) — stated in the limitations. LightGBM excluded: its SynapseML native library is x86_64-only, and the deployment target is ARM64.

Backup: training and prediction cost

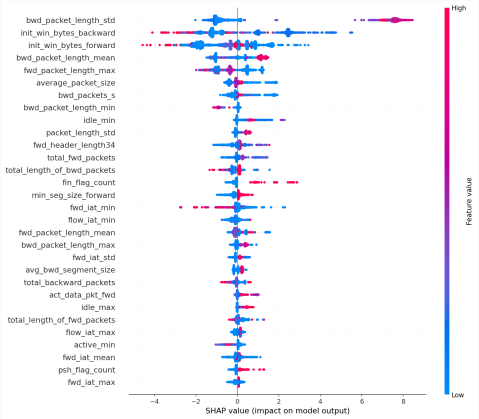


Measured on the Spark training cluster. Latency/throughput *on the edge device* belong to the companion systems paper.

Backup: threats to validity

- **Internal.** RF/SHAP rankings come from the original training set and stay fixed across resamplings — consistent with the a-priori design, but a slight leak in the ranking step. Re-ranking per resampling is more rigorous, but costly for SHAP. MLP carries the unweighted-training caveat.
- **External.** CICIDS2017 may not reflect current traffic, and benchmark NIDS datasets have documented quality limitations. The cross-dataset conclusion rests on a *stratified sample* of CSE-CIC-IDS2018. Evasive traffic untested. Feature-level dedup itself shifts the benign/attack distribution.
- **Statistical.** $N{=}6$ gives low power: two-sided exact enumeration floors p at $2/2^N$, so $N{=}6$ *just* reaches 0.05. Resamplings overlap, violating t-independence — hence Nadeau–Bengio as the primary basis. Nested CV with larger N is the next reinforcement.

Backup: SHAP feature attribution



SHAP is computed on a *class-stratified* sample so both benign and attack traffic are represented in the ranking.